



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**Design and Implementation of an Efficient Priority-Based
Emergency Patient Management System Using Appropriate Data
Structures for Severity-Based Treatment and Arrival-Order
Processing**

Submitted in the partial fulfilment for the Course
of

CSA0307 - Data Structures

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

B.TECH – IT

Submitted by

Jaffrin N (192521200)

Under the Supervision of
Dr. K. Kiruthika

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences,
Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.Tech
Course Code & Course Name	CSA0307 Data Structure
Academic Year / Batch	2 nd year (2025-2029)
Faculty Name	Dr.K.Kiruthika
Assignment Title	Design and Implementation of an Efficient Priority-Based Emergency Patient Management System Using Appropriate Data Structures for Severity-Based Treatment and Arrival-Order Processing
Date of Issue	31-08-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Select and apply a suitable Abstract Data Type. CO2: Apply Queue and Priority Queue for severity-based treatment while maintaining arrival order among patients with the same severity. CO4: Implement an efficient priority-based emergency patient management system using a Priority Queue and evaluate time and space complexity.
Bloom's Taxonomy Level	L4 – Analyze; L5 – Evaluate
SDG Mapping (SDG 1-17)	SDG-3, SDG-9 and SDG-11
Industry / Societal Relevance	SDG 3 – Good Health and Well-being; SDG 9 – Industry, Innovation and Infrastructure; SDG 10 – Reduced Inequalities; SDG 11 – Sustainable Cities and Communities.

A. Assignment Problem / Challenge

Design an efficient Priority-Based Emergency Patient Management System for a hospital emergency department that continuously receives patients with different levels of severity. The system uses a Priority Queue implemented with a Max-Heap to provide severity-based treatment while maintaining arrival order among patients with the same severity. It supports patient registration, priority assignment, next-patient selection, treatment/removal, searching, updating and displaying patient information, and is evaluated using performance, memory usage, fairness, responsiveness, reliability and scalability.

C. Problem Statement

Problem / Case / Design Challenge:

A hospital emergency department receives patients continuously throughout the day, and each patient may require a different level of medical attention depending on the severity of the condition. For every patient, the system maintains a unique Patient ID, patient name, arrival number or arrival time, severity level and assigned doctor. Critical or life-threatening patients must be treated before less severe patients. When two or more patients have the same severity, they must be treated in the order in which they arrived. A simple FIFO queue cannot satisfy this requirement because an earlier low-severity patient may block a later critical patient. Therefore, the system requires a suitable Priority Queue that combines severity-based priority with FIFO processing for equal-severity patients. The complete C-based implementation supports registration, priority assignment, selection of the next patient, treatment/removal, searching, updating and displaying patient records.

D. Requirements and Constraints

- 1 Functional: Register and store patient details.
- 2 Performance: Select the highest-priority patient efficiently.
- 3 Technical: Use a Priority Queue implemented with a Max-Heap.
- 4 Safety: Critical and life-threatening patients must be considered first.
- 5 Reliability: Maintain FIFO order among patients with the same severity.
- 6 User: Provide a simple menu-driven interface.
- 7 Security: Handle patient information carefully.

E. Student Work

Problem Understanding and Formulation

1.1 What is the problem?

The emergency department must continuously decide which patient should be treated next. Since patient severity is not the same for everyone, treatment cannot be based only on arrival order. The proposed system assigns each patient a severity level from 1 to 5 and uses a Priority Queue implemented with a Max-Heap. Higher severity receives higher priority. If two patients have the same severity, the patient with the smaller arrival number is selected first. This combines emergency severity with fairness of arrival order.

The Max-Heap root always represents the patient who should be treated next. After treatment, the root is removed and the heap property is restored. This provides efficient priority-based processing while keeping the implementation suitable for a C program.

1.2 Expected outcomes

Register and store patient details.

Assign severity from 1 to 5.

Insert patients into the Max-Heap Priority Queue.

Select the highest-priority patient for treatment.

Maintain FIFO order for equal severity.

Search by Patient ID.

Update patient information and re-adjust priority.

1.3 Remove treated patients and maintain treatment history.

Data	Description
Patient ID	Unique identifier for each patient
Patient Name	Name of the patient
Arrival Number	Sequential number used to maintain FIFO order
Arrival Time	Time of entry into emergency department

Severity Level	1–5 priority level based on urgency
Severity Description	5 Critical, 4 Severe, 3 Moderate, 2 Mild, 1 Stable
Assigned Doctor	Doctor responsible for the patient
Treatment Status	Waiting, In Treatment or Treated

1.4 Assumptions

1. Patient IDs are unique.
2. Arrival numbers increase sequentially.
3. Severity levels range from 1 to 5.
4. Higher severity means higher priority.
5. Equal severity uses earlier arrival first.
6. The Max-Heap root contains the highest-priority patient.
7. A patient is removed after treatment.
8. The prototype does not replace clinical decision-making.

1.5 Constraints

- The priority rule must always be preserved.
- Patient IDs must remain unique.
- The system must support priority changes.
- Treatment must remove the current highest-priority patient.
- Search may require linear time.
- Memory should remain proportional to waiting patients.

2. Application of Course Knowledge

2.1 Abstract Data Type and Patient Structure

A Patient ADT stores Patient ID, patient name, arrival number, arrival time, severity level, assigned doctor and treatment status. A C structure groups these related fields into one patient record.

2.2 Queue Data Structure

A normal Queue follows FIFO order and is useful when all patients have the same priority. It is not sufficient for an emergency department because a critical patient arriving later must move ahead of

a less severe patient. Therefore, the Queue concept is retained for arrival-order reasoning, while the Priority Queue is used for treatment selection.

2.3 Priority Queue

Severity levels are defined as 5 – Critical, 4 – Severe, 3 – Moderate, 2 – Mild and 1 – Stable. Higher severity receives higher priority. When severity values are equal, the smaller arrival number receives higher priority.

2.4 Max-Heap

A Max-Heap implements the Priority Queue. During insertion, the new patient is added at the end and moved upward using heapify-up. During treatment, the root patient is removed, the last patient is moved to the root and heapify-down restores the heap property.

2.5 Priority Comparison Logic

The comparison rule first checks severity. If severity is equal, arrival number is checked. Example: P101(2,1), P102(5,2), P103(3,3), P104(5,4), P105(2,5) gives treatment order P102 → P104 → P103 → P101 → P105.

2.6 Insertion

- A new patient is inserted at the next available heap position and heapify-up is performed. Worst-case time: $O(\log n)$.

2.7 Treatment / Deletion

- The root is selected for treatment, replaced by the last heap element, and heapify-down restores the Max-Heap. Time: $O(\log n)$.

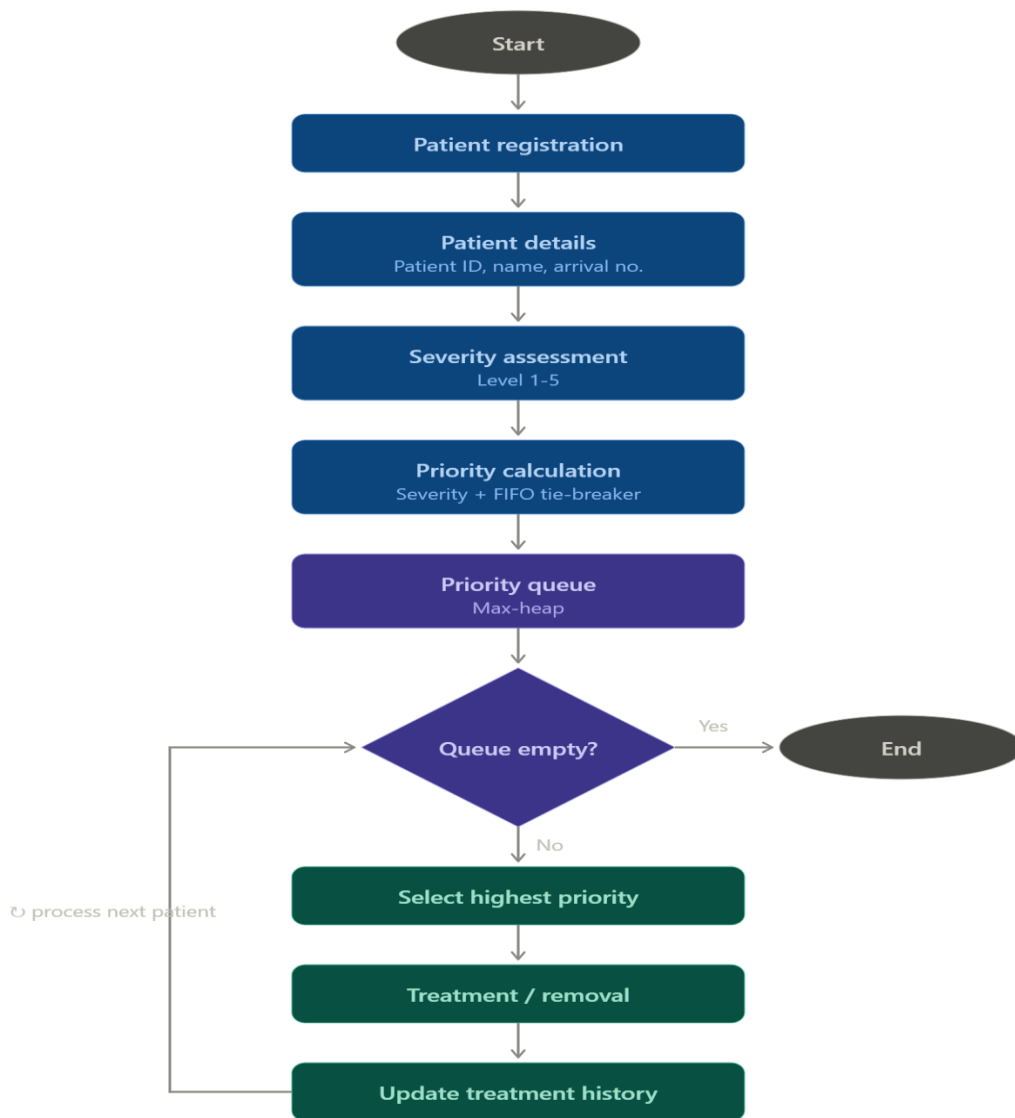
2.8 Search and Update

Searching by Patient ID takes $O(n)$. Updating may take $O(n)$ to locate the patient plus $O(\log n)$ when priority changes and reheapification is required.

2.9 Complexity

Insertion $O(\log n)$ | Delete highest $O(\log n)$ | Peek $O(1)$ | Search $O(n)$ | Update $O(n)+O(\log n)$ when priority changes | Display $O(n)$ | Space $O(n)$.

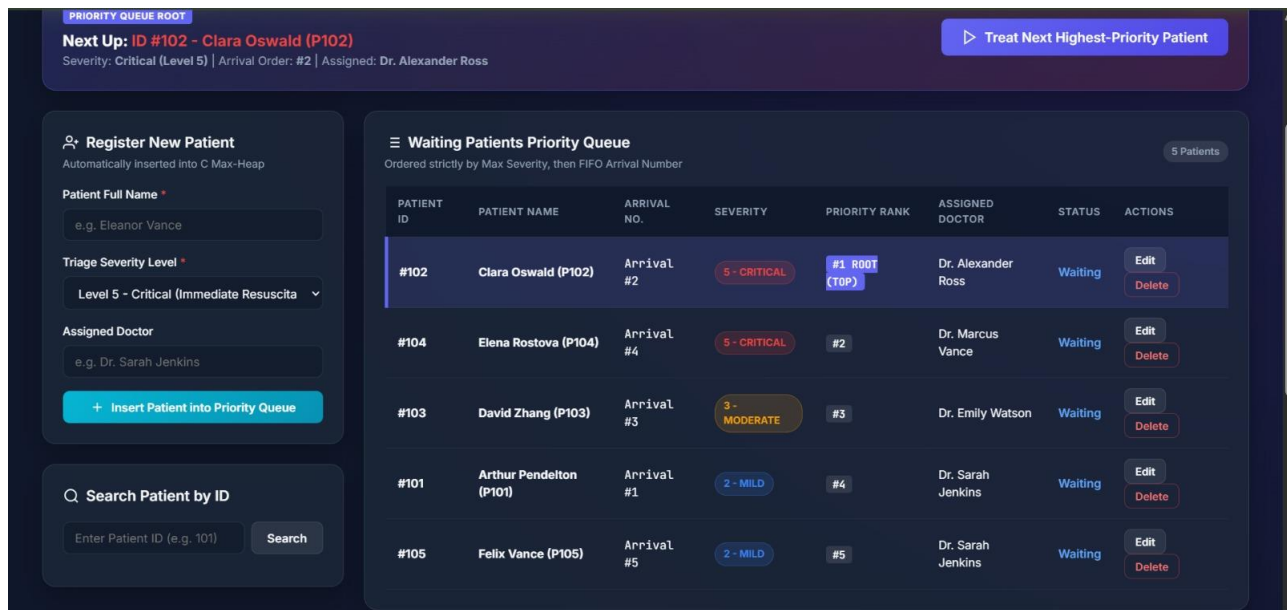
3. Solution / Design / Methodology



4. Use of Modern Tools

C is used for the core implementation. GCC compiles and executes the program, while Visual Studio Code can be used for development and debugging. C structures model patient records, arrays store the heap, standard I/O supports the menu, and file handling can be used for treatment history.

5. Results and Validation



```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#include "patient.h"
```

```
#include "log.h"
```

```
#include "server.h"
```

```
// Simulation worker threads (Registration Counters & Treatment Departments)
```

```
void* registrationCounterThread(void *arg) {
```

```
    int counterId = (int)(intptr_t)arg;
```

```
    const char *sampleNames[] = {"Arthur Pendelton", "Clara Oswald", "David Zhang",  
    "Elena Rostova", "Felix Vance"};
```

```
    const char *sampleSymptoms[] = {"Acute respiratory distress", "Severe migraine and  
    dizziness", "Blunt force trauma", "Deep laceration", "High fever"};
```

```
    int count = sizeof(sampleNames) / sizeof(sampleNames[0]);
```

```

    for (int i = 0; i < count; i++) {
sleep(8 + counterId * 2 + i * 5); // Staggered registration activity
int level = (i % 5) + 1;
int id = registerPatient(sampleNames[i], level, sampleSymptoms[i]);

printf("[SIMULATION] Counter %d registered Patient ID=%d (%s)\n", counterId, id,
sampleNames[i]);

    }

    return NULL;
}

```

```

void* treatmentDepartmentThread(void *arg) {
    int deptId = (int)(intptr_t)arg;

    const char *doctors[] = {"Dr. House", "Dr. Strange", "Dr. Grey", "Dr. McCoy"};
    int docCount = sizeof(doctors) / sizeof(doctors[0]);

    while (1) {
        sleep(10 + deptId * 3);

        // Periodically update patient status to demonstrate background concurrency
        // Pick a patient ID between 1001 and 1010
        int targetId = 1001 + (rand() % 8);

        Patient *p = searchPatient(targetId);

        if (p) {
            const char *doc = doctors[rand() % docCount];

            updatePatient(targetId, IN_TREATMENT, doc);

            printf("[SIMULATION] Department %d assigned %s to Patient ID=%d\n",
deptId, doc, targetId);

        }
    }
}

```

```

        return NULL;
    }

    int main(void) {

        printf("=====\\n
        ");

        printf("  SMART EMERGENCY PATIENT COORDINATION SYSTEM (C)  \\n");

        printf("=====\\n
        ");
    }

```

Test Cases

Case 1 – Normal Patient Registration:

- Patient: P101 with severity 2.
- Condition: No other patients are waiting.
- Result: P101 is inserted successfully into the priority queue

The screenshot displays a medical emergency system interface. At the top, a status bar indicates the next patient to be treated: ID #102 - Clara Oswald (P102), with a severity of Critical (Level 5), arrival order of #2, and assigned to Dr. Alexander Ross. A button labeled 'Treat Next Highest-Priority Patient' is visible.

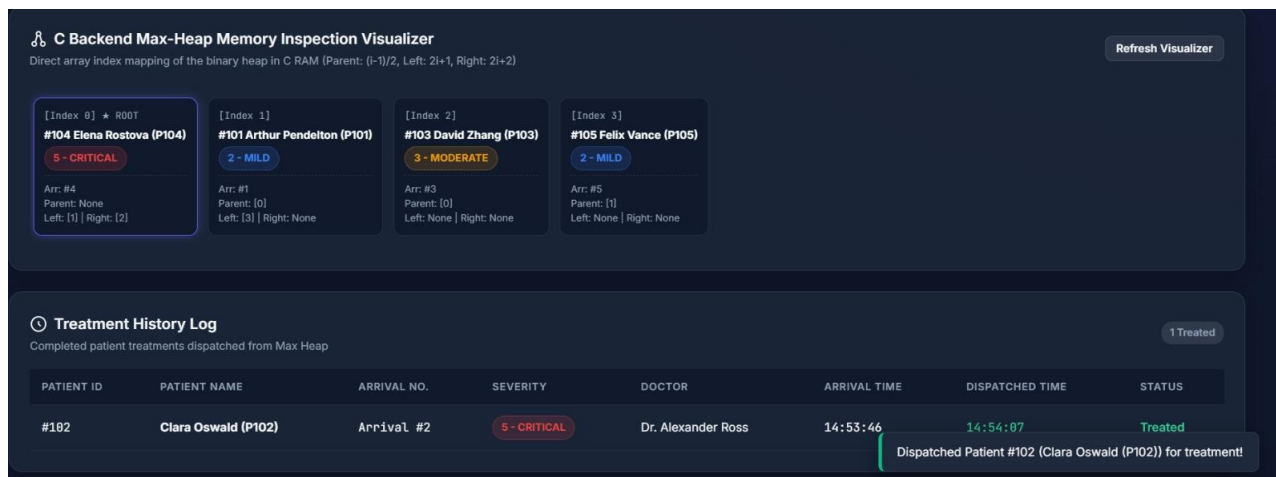
On the left, there is a 'Register New Patient' form with fields for Patient Full Name, Triage Severity Level (set to Level 5 - Critical), and Assigned Doctor (Dr. Sarah Jenkins). A button 'Insert Patient into Priority Queue' is at the bottom of the form.

The main section is titled 'Waiting Patients Priority Queue' and shows a table of patients waiting for treatment, ordered by Max Severity and then FIFO Arrival Number. The table has 5 patients.

PATIENT ID	PATIENT NAME	ARRIVAL NO.	SEVERITY	PRIORITY RANK	ASSIGNED DOCTOR	STATUS	ACTIONS
#102	Clara Oswald (P102)	Arrival #2	5 - CRITICAL	#1 ROOT (10P)	Dr. Alexander Ross	Waiting	Edit Delete
#104	Elena Rostova (P104)	Arrival #4	5 - CRITICAL	#2	Dr. Marcus Vance	Waiting	Edit Delete
#103	David Zhang (P103)	Arrival #3	3 - MODERATE	#3	Dr. Emily Watson	Waiting	Edit Delete
#101	Arthur Pendelton (P101)	Arrival #1	2 - MILD	#4	Dr. Sarah Jenkins	Waiting	Edit Delete
#105	Felix Vance (P105)	Arrival #5	2 - MILD	#5	Dr. Sarah Jenkins	Waiting	Edit Delete

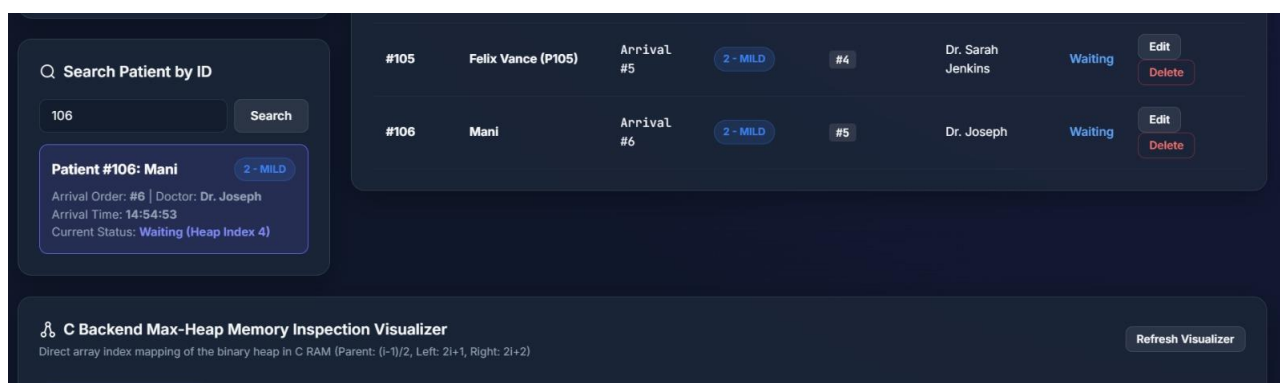
Case 2 – Critical Patient Arrives Later:

- **Patients:** P101 with severity 2 and P102 with severity 5.
- **Condition:** P102 arrives after P101 but has higher severity.
- **Result:** P102 is selected first for treatment.



Case 3 – Same Severity FIFO Processing:

- **Patients:** P101 and P102 both have severity 4.
- **Condition:** P101 arrives before P102.
- **Result:** P101 is treated first, following FIFO arrival order.



6. Analysis and Engineering Decision

Final Engineering Decision

A Priority Queue implemented with a Max-Heap was selected because the emergency department repeatedly needs the highest-severity patient. A simple Queue cannot prioritize critical cases. An array-based or linked-list priority queue may require $O(n)$ scanning or insertion. The Max-Heap provides $O(\log n)$ insertion and deletion of the highest-priority patient with $O(n)$ memory.

Algorithm Comparison

Tool/Library	Purpose
C	Core implementation
GCC	Compilation and execution
Visual Studio Code	Development and debugging
C Standard Library	Input/output and utilities
Structures	Store patient records
Arrays	Store the Max-Heap
Max-Heap	Implement the Priority Queue
File Handling	Optional treatment-history storage

Simple Queue: simple FIFO but cannot prioritize critical patients. Array-Based Priority Queue: easy storage but may require $O(n)$ shifting/search. Linked-List Priority Queue: dynamic but priority insertion/search can be $O(n)$. Max-Heap Priority Queue: efficient priority insertion and highest-priority deletion; selected as the final solution.

7. Broader Considerations

7.1 Safety

The system prioritizes critical and life-threatening patients, reducing delays caused by simple arrival-order processing.

7.2 Society

Organized emergency-patient processing can improve responsiveness and fairness by considering severity while preserving arrival order for equal-severity patients.

7.3 Environment

Efficient patient flow can reduce unnecessary waiting and resource usage within emergency departments.

7.4 Economics

Better prioritization can improve utilization of doctors, emergency rooms and support resources.

7.5 Professional Responsibility

A real deployment would require validated data, secure patient information, extensive testing, fault handling and human clinical oversight. The software should support staff decisions rather than replace medical judgment.

8. Conclusion

The Priority-Based Emergency Patient Management System demonstrates how Data Structures and Algorithms can solve a practical hospital emergency problem. A Priority Queue implemented using a Max-Heap ensures that patients with greater severity are treated first, while arrival number provides FIFO ordering for equal-severity patients. The system supports registration, search, update, display, treatment and deletion. The design achieves $O(\log n)$ insertion and deletion, $O(1)$ access to the next patient, $O(n)$ search and $O(n)$ space. Future improvements can include database storage, a web interface, real-time hospital monitoring, multiple departments, role-based access and hospital-information-system integration.

9. Student Reflection

Through this assignment, I learned how queues, priority queues, structures and heaps can be applied to an emergency-management problem. I understood that a normal Queue is suitable when all items have equal priority, while a Priority Queue is better when urgency levels differ. I also learned how to combine severity and arrival order to create a fair priority rule. Implementing insertion, deletion, searching and updating improved my understanding of heap operations and time complexity.

10. References

- Improving Patient Flow in Emergency Department Through Dynamic Priority Queue,” IEEE Conference Publication.

- “Development of Automated Triage System for Emergency Medical Service,” IEEE Conference Publication.
- “Analysing the ED Patient Flow Management Problem by Using Accumulating Priority Queues and Simulation-Based Optimization,” IEEE Conference Publication.
- “Queue Model in Health Facilities Information System,” IEEE Conference Publication.
- “Implementation of Classification Algorithm C4.5 in Determining the Emergency Patient in the Maternity Hospital Queue System,” IEEE Conference Publication.

F. Common Assessment Rubric

G. CO–PO–Assessment Mapping

H. Faculty Evaluation & Continuous Improvement CO Attainment

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment

8. CO attainment analysis

9. Sample student work representing different performance levels

- 10. Faculty analysis and continuous-improvement action

-

-

-

-

-

-

-

•

-
-
-

1.

Algorithm	Advantage	Limitation	Suitability
Simple Queue	Very simple FIFO	Cannot prioritize critical patients	Low
Array-Based Priority Queue	Easy direct storage	May require $O(n)$ shifting/search	Medium
Linked-List Priority Queue	Dynamic memory	Priority insertion/search can be $O(n)$	Medium
Max-Heap Priority Queue	$O(\log n)$ insertion/deletion	Patient-ID search is $O(n)$	High

2.

2.1

2.2

2.3

2.4

2.5

3.

4.

5.

[1]

[2]

[3]

[4]

[5]

[6]

[7]

[8]

[9]

[10]

B.

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

C.

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1	PO1, PO2	L3/L4	10
Application of knowledge	CO1, CO2	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO2, CO4	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO4	PO5	L3/L4	10
Validation	CO2, CO4	PO4	L4/L5	15
Analysis & justification	CO4	PO2, PO3	L4/L5	15
Broader considerations	CO4	PO6, PO7	L4/L5	5
Documentation & reflection	CO1, CO4	PO10	L3/L4	5

D.

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.